เอกสารรายงานโปรเจกต์และข้อกำหนดความต้องการซอฟต์แวร์ (SRS & Project Report)

Simple Booking System for Lab Room

ระบบจองห้องปฏิบัติการ สำหรับสาขาวิชาวิศวกรรมคอมพิวเตอร์

สาขาวิชาวิศวกรรมคอมพิวเตอร์ มหาวิทยาลัยกาฬสินธุ์ (Kalasin University)

ผู้จัดทำ	**นาย ภคิน ภารสมบูรณ์**
อาจารย์ประจำวิชา	**อาจารย์ วรพจน์ สมมูล**
สาขาวิชา	วิศวกรรมคอมพิวเตอร์ (Computer Engineering)
สถาบันการศึกษา	มหาวิทยาลัยกาฬสินธุ์ (Kalasin University)
เทคโนโลยีที่ใช้	HTML5, CSS3, JavaScript (ES Modules), IndexedDB, Chart.js
GitHub Repository	https://github.com/Newphakhin/esfsdfsdf

รายงานนี้เป็นส่วนหนึ่งของการพัฒนาเว็บแอปพลิเคชันตามข้อกำหนด SRS (โปรเจกต์ระยะเวลา 2 สัปดาห์)

สาขาวิชาวิศวกรรมคอมพิวเตอร์ มหาวิทยาลัยกาฬสินธุ์

ปีการศึกษา 2569

EXECUTIVE SUMMARY & TABLE OF CONTENTS

บทคัดย่อ และสารบัญรายงาน

บทคัดย่อ (Abstract)

โปรเจกต์ **Simple Booking System for Lab Room (ระบบจองห้องปฏิบัติการ สำหรับสาขาวิชาวิศวกรรมคอมพิวเตอร์ มหาวิทยาลัยกาฬสินธุ์)** ได้รับการออกแบบและพัฒนาขึ้นเพื่อแก้ไขปัญหาการจองห้องปฏิบัติการซ้ำซ้อน การจดบันทึกด้วยเอกสารกระดาษหรือการแจ้งผ่านช่องทางแชทที่ไม่เป็นระบบ โดยพัฒนาเป็นเว็บแอปพลิเคชันแบบ Single Page Application (SPA) ด้วยเทคโนโลยี **HTML5, CSS3 และ Modern JavaScript (ES Modules)** ควบคู่กับการจัดเก็บข้อมูลในเบราว์เซอร์ด้วยฐานข้อมูล **IndexedDB (`LabBookingDB`)** สำหรับข้อมูลห้องและรายการจอง และ **LocalStorage** สำหรับจัดการเซสชันผู้ใช้และสิทธิ์ของผู้ดูแลระบบ

จุดเด่นของระบบคือการแยกโครงสร้างโค้ดออกเป็น 3 ชั้นอย่างชัดเจน ได้แก่ ชั้นส่วนติดต่อผู้ใช้ (Pages & Components), ชั้นบริการจัดการข้อมูล (Services) และชั้นกฎธุรกิจ (Domain Rules) โดยเฉพาะอัลกอริทึมตรวจสอบช่วงเวลาซ้ำซ้อน (`new_start_time < existing_end_time AND new_end_time > existing_start_time`) ที่ทำงานทั้งแบบ Real-time บนหน้าแบบฟอร์มจอง และตรวจสอบซ้ำในขั้นตอนการอนุมัติของผู้ดูแลระบบ ช่วยป้องกันปัญหาการอนุมัติห้องซ้ำซ้อนได้อย่างสมบูรณ์ นอกจากนี้ระบบยังผ่านการทดสอบด้วยชุดทดสอบอัตโนมัติ (Unit Tests) จำนวน 48 ชุดทดสอบ ครอบคลุมทุกเกณฑ์การตรวจรับ (Acceptance Criteria) ตามเอกสาร SRS ทุกประการ

สารบัญเนื้อหา (Table of Contents)

บทที่ 1 บทนำและข้อกำหนดความต้องการของระบบ (SRS) — **ส่วนที่ 1**

- 1.1 ชื่อโปรเจกต์และภาพรวมของระบบ (Project Overview) — 1.1
- 1.2 วัตถุประสงค์ของระบบ และกลุ่มผู้ใช้งาน — 1.2
- 1.3 ขอบเขตของระบบ (ขอบเขตหลัก ขอบเขตเสริม และสิ่งที่ไม่อยู่ในขอบเขต) — 1.3
- 1.4 ความต้องการเชิงฟังก์ชัน (Functional Requirements: FR-01 ถึง FR-18) — 1.4
- 1.5 ความต้องการไม่เชิงฟังก์ชัน (Non-Functional Requirements: NFR-01 ถึง NFR-08) — 1.5
- 1.6 แผนการดำเนินงาน 2 สัปดาห์ (Day 1 – Day 14) — 1.6

บทที่ 2 การวิเคราะห์และออกแบบระบบ (System Analysis & Design) — **ส่วนที่ 2**

- 2.1 สถาปัตยกรรมระบบแบบแยกชั้น (Layered SPA Architecture) — 2.1
- 2.2 การออกแบบฐานข้อมูลและพจนานุกรมข้อมูล (Lab Room, Booking, Session) — 2.2
- 2.3 สถานะของการจอง (Booking Status) และกฎการจองเบื้องต้น — 2.3
- 2.4 อัลกอริทึมและเงื่อนไขการตรวจสอบเวลาซ้ำซ้อน (Time Overlap Logic) — 2.4

2.5 ขั้นตอนการทำงานของระบบ (Workflow การจอง การอนุมัติ และการดูตาราง) 2.5

บทที่ 3 การพัฒนาระบบและอธิบายโค้ดหลัก (System Implementation) **ส่วนที่ 3**

3.1 เทคโนโลยีที่ใช้พัฒนา (Technology Stack) และโครงสร้างโปรเจกต์ 3.1

3.2 อธิบายโค้ดหลัก: ฐานข้อมูล IndexedDB (`js/db.js`) 3.2

3.3 อธิบายโค้ดหลัก: กฎธุรกิจและตรวจเวลาทับซ้อน (`js/domain/rules.js`) 3.3

3.4 อธิบายโค้ดหลัก: บริการจัดการการจองและห้อง (`bookingService.js` , `roomService.js`) 3.4

3.5 ข้อมูลตัวอย่างในระบบ (ห้องปฏิบัติการ 5 ห้อง และการจอง 16 รายการ) 3.5

บทที่ 4 การทดสอบระบบและคู่มือการใช้งานพร้อมภาพหน้าจอ **ส่วนที่ 4**

4.1 ผลการทดสอบระดับหน่วย (Unit Testing: 48 ชุดทดสอบ) 4.1

4.2 การตรวจสอบตามเกณฑ์ตรวจรับระบบ (Acceptance Criteria 13 ข้อ) 4.2

4.3 ภาพหน้าจอหลักและคู่มือการใช้งานครบทั้ง 10 หน้าจอ 4.3

บทที่ 5 สรุปผลการดำเนินงาน การใช้ AI Tools และแนวทางการ Deploy **ส่วนที่ 5**

5.1 สรุปผลการพัฒนาเทียบกับ MVP และผลลัพธ์การเรียนรู้ (Learning Outcomes) 5.1

5.2 สรุปการใช้ AI Tools ในการพัฒนา (ตามข้อกำหนด SRS ข้อ 22–23) 5.2

5.3 ขั้นตอนการติดตั้ง การใช้งาน และการนำขึ้นระบบจริง (GitHub & Deployment) 5.3

CHAPTER 1 · INTRODUCTION & SOFTWARE REQUIREMENTS SPECIFICATION

บทที่ 1 บทนำและข้อกำหนดความต้องการของระบบ (SRS)

1.1 ชื่อโปรเจกต์และภาพรวมของระบบ (SRS ข้อ 1–2)

- **ชื่อโปรเจกต์ (ภาษาอังกฤษ):** Simple Booking System for Lab Room
- **ชื่อโปรเจกต์ (ภาษาไทย):** ระบบจองห้องปฏิบัติการสำหรับสาขาวิชาวิศวกรรมคอมพิวเตอร์ มหาวิทยาลัยกาฬสินธุ์

ภาพรวมของระบบ: Simple Booking System for Lab Room เป็นเว็บแอปพลิเคชันสำหรับจัดการการจองห้องปฏิบัติการของสาขาวิชาวิศวกรรมคอมพิวเตอร์ มหาวิทยาลัยกาฬสินธุ์ โดยมีวัตถุประสงค์เพื่อให้ผู้ใช้งาน เช่น นักศึกษา อาจารย์ หรือเจ้าหน้าที่ สามารถตรวจสอบห้องว่าง ส่งคำขอจองห้อง และติดตามสถานะการจองได้อย่างสะดวก ระบบนี้ช่วยลดปัญหาการจองซ้ำซ้อน การจดบันทึกด้วยเอกสารหรือช่องทางแชทที่ไม่เป็นระบบ และช่วยให้ผู้ดูแลสามารถตรวจสอบรายการจอง อนุมัติหรือปฏิเสธคำขอจองได้ง่ายขึ้น

1.2 วัตถุประสงค์ของระบบ (SRS ข้อ 3)

1. เพื่อพัฒนาเว็บแอปพลิเคชันสำหรับจองห้องปฏิบัติการของสาขาวิชาวิศวกรรมคอมพิวเตอร์
2. เพื่อให้ผู้ใช้สามารถตรวจสอบรายการจองและสถานะห้องได้อย่างสะดวก
3. เพื่อป้องกันการจองห้องซ้ำซ้อนในวันและเวลาเดียวกัน
4. เพื่อให้ผู้ดูแลระบบสามารถอนุมัติ ยกเลิก หรือปฏิเสธคำขอจองได้
5. เพื่อฝึกให้นักศึกษาพัฒนาเว็บแอปพลิเคชันที่มี CRUD, validation, role-based view, calendar/table view และ dashboard อย่างง่าย
6. เพื่อส่งเสริมให้นักศึกษาใช้ AI tools ช่วยวางโครงสร้างระบบ เขียนโค้ด แก้บั๊ก และจัดทำเอกสารประกอบโปรเจกต์

1.3 กลุ่มผู้ใช้งานและบทบาทผู้ใช้งาน (SRS ข้อ 4 และ ข้อ 7)

กลุ่มผู้ใช้งานหลักของระบบประกอบด้วย 4 กลุ่ม ได้แก่ (1) นักศึกษาสาขาวิชาวิศวกรรมคอมพิวเตอร์, (2) อาจารย์ประจำสาขาวิชาวิศวกรรมคอมพิวเตอร์, (3) เจ้าหน้าที่หรือผู้ดูแลห้องปฏิบัติการ และ (4) ผู้ดูแลระบบ โดยแบ่งสิทธิ์การใช้งานออกเป็น 2 บทบาทหลัก ดังนี้:

7.1 บทบาทผู้ใช้งานทั่วไป (User)

- ดูรายการห้องปฏิบัติการทั้งหมดที่เปิดให้จอง
- ดูรายละเอียดห้อง (ความจุ อุปกรณ์ สถานที่ สถานะ)
- ตรวจสอบตารางการจองรายวันและรายสัปดาห์
- ส่งคำขอจองห้องพร้อมตรวจสอบเวลาซ้ำซ้อน
- ดูสถานะคำขอจองของตนเอง (แยกตามอีเมลผู้จอง)
- ยกเลิกคำขอจองของตนเอง หากยังไม่ถึงเวลาที่กำหนด

7.2 บทบาทผู้ดูแลระบบ (Admin)

- เข้าสู่ระบบผู้ดูแลผ่านหน้า Login (`#/login`)
- เพิ่ม แก้ไข และลบข้อมูลห้องปฏิบัติการ (CRUD)
- ดูรายการคำขอจองทั้งหมดในระบบ
- อนุมัติคำขอจอง (พร้อมตรวจซ้ำซ้อนอัตโนมัติ)
- ปฏิเสธคำขอจองพร้อมระบุเหตุผลประกอบ
- ยกเลิกรายการจอง และค้นหา/กรองรายการจอง
- ดู Dashboard สรุปข้อมูลสถิติการใช้ห้องปฏิบัติการ

1.4 ขอบเขตของระบบ (SRS ข้อ 5–6)

1.4.1 ขอบเขตหลักที่ต้องทำ (Core Scope — ดำเนินการครบ 100%)

1. แสดงรายการห้องปฏิบัติการที่สามารถจองได้
2. แสดงรายละเอียดของห้อง เช่น ชื่อห้อง ความจุ อุปกรณ์ และสถานที่
3. ผู้ใช้สามารถส่งคำขอจองห้องได้
4. ผู้ใช้สามารถระบุวันที่ เวลาเริ่มต้น เวลาสิ้นสุด วัตถุประสงค์ และจำนวนผู้ใช้งานได้
5. ระบบตรวจสอบการจองซ้ำซ้อนในห้องเดียวกัน วันเดียวกัน และช่วงเวลาเดียวกันได้
6. ผู้ใช้สามารถดูรายการจองของตนเองได้
7. ผู้ดูแลสามารถดูคำขอจองทั้งหมดได้
8. ผู้ดูแลสามารถอนุมัติ ปฏิเสธ หรือยกเลิกคำขอจองได้
9. ระบบสามารถแสดงตารางการจองรายวันหรือรายสัปดาห์ได้
10. ระบบสามารถค้นหาและกรองรายการจองตามห้อง วันที่ หรือสถานะได้
11. ระบบสามารถแสดง Dashboard สรุปจำนวนการจองตามสถานะได้
12. Deploy ระบบให้สามารถใช้งานผ่าน URL ได้จริง

1.4.2 ขอบเขตเสริมที่ได้พัฒนาเพิ่มเติมในโปรเจกต์นี้ (Implemented Optional Features)

- **ระบบ Login แยกบทบาท User และ Admin:** พัฒนาหน้า `#/login` พร้อมระบบป้องกันการเข้าถึงหน้า Admin (Route Guard `adminOnly` ตาม NFR-05)
- **ตารางการจองแบบ Visual Timeline (Day & Week Calendar View):** คำนวณความสูงบล็อกตามเวลาจริงและจัดเรียงบล็อกที่เวลาซ้อนกันเป็นคอลัมน์ย่อย (Multi-lane layout)
- **โทนสี Dark Mode ระดับพรีเมียม:** ออกแบบด้วย CSS Variables โทนสีเข้ม ลดความล้าของสายตาและอ่านง่ายในทุกขนาดหน้าจอ

1.4.3 สิ่งที่ไม่อยู่ในขอบเขต (Out of Scope สำหรับโปรเจกต์ 2 สัปดาห์)

ไม่เชื่อมต่อระบบทะเบียนนักศึกษาจริงหรือ LDAP/SSO ของมหาวิทยาลัย, ไม่ทำระบบอนุมัติหลายระดับ, ไม่คิดค่าบริการการใช้ห้อง, ไม่ทำ Native Mobile App, ไม่เชื่อมต่อ Google Calendar จริง และไม่ควบคุมอุปกรณ์ IoT หรือประตูห้องจริง

1.5 ความต้องการเชิงฟังก์ชัน (Functional Requirements — SRS ข้อ 8)

ตารางที่ 1.1 รายการความต้องการเชิงฟังก์ชัน (FR-01 ถึง FR-18) และการจับคู่กับโมดูลในระบบจริง

รหัส	รายการความต้องการ	รายละเอียดตาม SRS	หน้าจอ / ไฟล์ที่รับผิดชอบ
FR-01	แสดงรายการห้องปฏิบัติการ	แสดงรายการห้องที่สามารถจองได้ เช่น CE Lab 1, CE Lab 2, Network Lab, IoT Lab	`#/rooms` `js/pages/roomList.js`
FR-02	แสดงรายละเอียดห้อง	แสดงชื่อห้อง ความจุ อุปกรณ์ สถานที่ และสถานะการใช้งาน	`#/rooms/:id` `js/pages/roomDetail.js`
FR-03	เพิ่มข้อมูลห้อง	Admin สามารถเพิ่มข้อมูลห้องปฏิบัติการใหม่ได้	`#/admin/rooms` `js/services/roomService.js`
FR-04	แก้ไขข้อมูลห้อง	Admin สามารถแก้ไขรายละเอียดห้องปฏิบัติการได้	`#/admin/rooms` `js/pages/adminRooms.js`
FR-05	ลบข้อมูลห้อง	Admin สามารถลบข้อมูลห้องที่ไม่ต้องการได้ (ป้องกันการลบหากมีการจองค้างอยู่)	`#/admin/rooms` `roomService.remove()`
FR-06	ส่งคำขอจองห้อง	User เลือกห้อง วันที่ เวลาเริ่มต้น เวลาสิ้นสุด วัตถุประสงค์ และจำนวนผู้ใช้งานได้	`#/book` `js/pages/bookingForm.js`
FR-07	ตรวจสอบเวลาซ้ำซ้อน	ตรวจสอบว่าห้องเดียวกันไม่ถูกจองซ้ำในช่วงเวลาที่ทับซ้อนกันกับรายการที่ Approved	`js/domain/rules.js` (`isOverlap` , `findConflicts`)
FR-08	บันทึกคำขอจอง	ระบบบันทึกคำขอจองลงฐานข้อมูล พร้อมสถานะเริ่มต้นเป็น `Pending`	`bookingService.create()` `js/db.js` (IndexedDB)
FR-09	ดูรายการจองของผู้ใช้	User สามารถดูรายการจองที่ตนเองส่งคำขอได้	`#/my` `js/pages/myBookings.js`
FR-10	ดูคำขอจองทั้งหมด	Admin สามารถดูคำขอจองทั้งหมดในระบบได้	`#/admin/bookings` `js/pages/adminBookings.js`
FR-11	อนุมัติคำขอจอง	Admin สามารถเปลี่ยนสถานะคำขอจองเป็น `Approved` ได้	`bookingService.approve()`
FR-12	ปฏิเสธคำขอจอง	Admin เปลี่ยนสถานะคำขอจองเป็น `Rejected` พร้อมระบุเหตุผลได้	`bookingService.reject()`
FR-13	ยกเลิกคำขอจอง	User หรือ Admin สามารถยกเลิกคำขอจองได้ตามเงื่อนไขที่กำหนด	`bookingService.cancel()` `canUserCancel()`

รหัส	รายการความต้องการ	รายละเอียดตาม SRS	หน้าจอ / ไฟล์ที่รับผิดชอบ
FR-14	แสดงตารางการจอง	แสดงรายการจองตามวันหรือสัปดาห์ในรูปแบบตารางเวลา	`#/schedule` `js/pages/schedule.js`
FR-15	ค้นหาและกรองรายการจอง	ค้นหาและกรองตามห้อง วันที่ ผู้จอง หรือสถานะได้	`bookingService.search()`
FR-16	Dashboard สรุปข้อมูล	แสดงจำนวนการจองทั้งหมด รออนุมัติ อนุมัติแล้ว ปฏิเสธ และยกเลิก พร้อมกราฟ	`#/admin/dashboard` `js/pages/dashboard.js`
FR-17	Responsive Design	แสดงผลได้ดีบนคอมพิวเตอร์ แท็บเล็ต และมือถือ	`css/style.css`
FR-18	Deploy ระบบ	สามารถเปิดใช้งานผ่าน URL สาธารณะ และเก็บโค้ดบน GitHub	GitHub / Static Hosting

1.6 ความต้องการไม่เชิงฟังก์ชัน (Non-Functional Requirements — SRS ข้อ 9)

ตารางที่ 1.2 รายการความต้องการไม่เชิงฟังก์ชัน (NFR-01 ถึง NFR-08) และแนวทางการออกแบบในระบบ

รหัส	หัวข้อ	รายละเอียดความต้องการ	การออกแบบในระบบจริง
NFR-01	ใช้งานง่าย	หน้าจอเรียบง่าย เมนูชัดเจน ใช้งานสะดวก	ใช้ Topbar แยกตามบทบาท มี Badge สถานะสีชัดเจน และแจ้งผลด้วย Toast Notification
NFR-02	ความถูกต้อง	ตรวจสอบการจองซ้ำซ้อนและคำนวณเวลาได้ถูกต้อง	แปลงเวลาเป็นนาที (`toMin`) และมี Unit Test ตรวจสอบ 29 กรณีทดสอบ
NFR-03	ความเร็ว	การค้นหา กรอง และบันทึกข้อมูลทำงานรวดเร็ว	ประมวลผลฝั่ง Client ด้วย IndexedDB และ DOM rendering แบบไม่ต้องรีโหลดหน้า
NFR-04	รองรับมือถือ	แสดงผลได้ดีทั้งบนคอมพิวเตอร์และสมาร์ตโฟน	ตารางข้อมูลเปลี่ยนเป็น Card อัตโนมัติบนหน้าจอแคบ พร้อมเมนู Hamburger
NFR-05	ความปลอดภัยเบื้องต้น	ป้องกันไม่ให้ User เข้าถึงหน้าจัดการของ Admin	ฟังก์ชัน `adminOnly()` ใน `app.js` บังคับ Redirect ไป `#/login` หากยังไม่ได้เข้าสู่ระบบ
NFR-06	ความเป็นส่วนตัว	ข้อมูลผู้จองถูกใช้เพื่อการจัดการจองห้องเท่านั้น	เก็บเฉพาะชื่อ อีเมล และประเภทผู้จองใน IndexedDB ของเบราว์เซอร์
NFR-07	ดูแลโค้ดง่าย	แบ่งโค้ดเป็น component, page, service, module ชัดเจน	แยกโฟลเดอร์ `domain/` , `services/` , `pages/` , `components/` , `utils/`
NFR-08	ความน่าเชื่อถือ	มี validation ป้องกันข้อมูลไม่ครบถ้วนหรือเวลาผิดพลาด	ฟังก์ชัน `validateBooking` และ `validateRoom` ตรวจสอบทุกฟิลด์ก่อนบันทึก

1.7 แผนการดำเนินงาน 2 สัปดาห์ (SRS ข้อ 21)

ตารางที่ 1.3 แผนการดำเนินงานพัฒนาโปรเจกต์ระยะเวลา 14 วัน (2 สัปดาห์)

วันที่	งานที่ดำเนินการ	วันที่	งานที่ดำเนินการ
Day 1	วิเคราะห์ Requirement จาก SRS และออกแบบโครงหน้าจอ	**Day 8**	พัฒนาหน้า Admin Booking Management (`adminBookings.js`)
Day 2	ออกแบบ Data Model (IndexedDB) และสร้าง Project Structure	**Day 9**	พัฒนาฟังก์ชันอนุมัติ ปฏิเสธ (พร้อมเหตุผล) และยกเลิกคำขอ
Day 3	สร้างข้อมูลห้องตัวอย่าง 5 ห้อง และหน้า Lab Room List	**Day 10**	พัฒนาหน้า Booking Schedule รายวันและรายสัปดาห์
Day 4	พัฒนาหน้า Lab Room Detail และหน้า Booking Form	**Day 11**	พัฒนาหน้า Dashboard (Chart.js) และระบบค้นหา/กรอง
Day 5	พัฒนาระบบบันทึกคำขอจองลงฐานข้อมูล (สถานะ `Pending`)	**Day 12**	ปรับแต่ง UI ให้เป็น Responsive และเขียน Unit Tests 48 ข้อ
Day 6	พัฒนาอัลกอริทึมตรวจสอบเวลาทับซ้อน (`isOverlap`)	**Day 13**	เตรียมระบบสำหรับ Deploy และอัปโหลดขึ้น GitHub Repository
Day 7	พัฒนาหน้า My Booking และระบบยกเลิกการจองของผู้ใช้	**Day 14**	จัดทำ README.md, บันทึกภาพหน้าจอ และจัดทำเล่มรายงานฉบับสมบูรณ์

CHAPTER 2 · SYSTEM ANALYSIS & DESIGN

บทที่ 2 การวิเคราะห์และออกแบบระบบ

2.1 สถาปัตยกรรมของระบบ (Layered Client-Side Architecture)

เพื่อให้สอดคล้องกับข้อกำหนด **NFR-07 (ดูแลโค้ดง่าย)** ระบบได้รับการออกแบบโดยแยกความรับผิดชอบออกเป็น 3 ชั้น (3-Layer Architecture) ด้วยมาตรฐาน ES Modules ดังนี้:

1. Presentation Layer (UI)

- `index.html` & `css/style.css`
- `js/app.js` & `js/router.js`
- `js/pages/*.js` (10 หน้าจอหลัก)
- `js/components/*.js` (Card & Modal)
- รับอินพุตจากผู้ใช้และเรนเดอร์ HTML

2. Service Layer (Application)

- `js/services/roomService.js`
- `js/services/bookingService.js`
- ประสานงานระหว่าง UI, กฎธุรกิจใน Domain และฐานข้อมูล
- ทำงานแบบ Asynchronous (`async/await`)

3. Domain & Data Layer

- `js/domain/rules.js` (Pure Functions)
- `js/domain/constants.js`
- `js/db.js` (IndexedDB: `LabBookingDB`)
- `js/session.js` (LocalStorage Session)
- `js/seed.js` (ข้อมูลตัวอย่างเริ่มต้น)

2.2 การออกแบบข้อมูลที่ต้องจัดเก็บ (Data Dictionary — SRS ข้อ 10)

2.2.1 โครงสร้างข้อมูลห้องปฏิบัติการ (10.1 Lab Room — Object Store: `rooms`)

ตารางที่ 2.1 พจนานุกรมข้อมูลตารางห้องปฏิบัติการ (`rooms`)

Field	ชนิดข้อมูล	รายละเอียดตาม SRS	ตัวอย่างข้อมูลจริงในระบบ
`id` (PK)	String	รหัสอ้างอิงห้องปฏิบัติการ (Primary Key)	`"room-ce1"`
`room_name`	String	ชื่อห้องปฏิบัติการ	`"Computer Engineering Lab 1"`
`room_code`	String (Unique)	รหัสห้องปฏิบัติการ (ห้ามซ้ำกัน)	`"CE-LAB-01"`
`location`	String	สถานที่ตั้ง (อาคาร / ชั้น / ห้อง)	`"อาคารวิศวกรรมศาสตร์ ชั้น 2 ห้อง 201"`
`capacity`	Number (Integer)	ความจุจำนวนผู้ใช้งานสูงสุด (คน)	`30`

Field	ชนิดข้อมูล	รายละเอียดตาม SRS	ตัวอย่างข้อมูลจริงในระบบ
`equipment`	Array<String>	รายการอุปกรณ์ประจำห้อง	`["Computer", "Projector", "Internet"]`
`description`	String	รายละเอียดเพิ่มเติมของห้อง	`"ห้องปฏิบัติการคอมพิวเตอร์ขนาดใหญ่..."`
`status`	String (Enum)	สถานะห้อง: `Available`, `Maintenance`, `Closed`	`"Available"`
`created_at`	String (ISO 8601)	วันและเวลาที่เพิ่มข้อมูลห้อง	`"2026-10-21T09:00:00.000Z"`

2.2.2 โครงสร้างข้อมูลการจองห้อง (10.2 Booking — Object Store: `bookings`)

ตารางที่ 2.2 พจนานุกรมข้อมูลตารางการจองห้องปฏิบัติการ (`bookings`)

Field	ชนิดข้อมูล	รายละเอียดตาม SRS	ตัวอย่างข้อมูลจริงในระบบ
`id` (PK)	String	รหัสการจอง (Primary Key)	`"bk-01"`
`room_id` (FK)	String (Indexed)	รหัสห้องที่จอง อ้างอิง `rooms.id`	`"room-ce1"`
`requester_name`	String	ชื่อ-นามสกุลผู้ขอจอง	`"สมชาย ใจดี"`
`requester_email`	String	อีเมลผู้จอง (ใช้ตรวจสอบสิทธิ์เจ้าของรายการ)	`"somchai@ksu.ac.th"`
`requester_type`	String (Enum)	ประเภทผู้จอง: `Student`, `Teacher`, `Staff`	`"Student"`
`booking_date`	String (Indexed)	วันที่จอง รูปแบบ `YYYY-MM-DD`	`"2026-10-21"`
`start_time`	String	เวลาเริ่มต้น รูปแบบ `HH:MM`	`"09:00"`
`end_time`	String	เวลาสิ้นสุด รูปแบบ `HH:MM`	`"11:00"`
`purpose`	String	วัตถุประสงค์การใช้งานห้อง	`"ติวสอบกลางภาควิชา Programming"`
`number_of_users`	Number (Integer)	จำนวนผู้เข้าใช้งาน (ต้องไม่เกิน `room.capacity`)	`20`
`status`	String (Indexed)	สถานะ: `Pending`, `Approved`, `Rejected`, `Cancelled`	`"Pending"`
`admin_comment`	String	ความเห็นหรือเหตุผลจากผู้ดูแลระบบ	`"ห้องถูกสำรองไว้สำหรับกิจกรรมคณะ"`

Field	ชนิดข้อมูล	รายละเอียดตาม SRS	ตัวอย่างข้อมูลจริงในระบบ
`created_at`	String (ISO 8601)	วันและเวลาที่ส่งคำขอจอง	`"2026-10-20T09:00:00.000Z"`
`updated_at`	String (ISO 8601)	วันและเวลาที่แก้ไขสถานะล่าสุด	`"2026-10-20T09:00:00.000Z"`

2.3 สถานะของการจองและกฎการจองเบื้องต้น (SRS ข้อ 12–13)

ระบบกำหนดสถานะของการจองออกเป็น 4 สถานะมาตรฐาน ได้แก่:

- Pending (รออนุมัติ) — สถานะเริ่มต้นเมื่อผู้ใช้ส่งคำขอจองใหม่ รอการพิจารณาจากผู้ดูแลระบบ
- Approved (อนุมัติแล้ว) — ผู้ดูแลอนุมัติให้ใช้งานห้องได้ และระบบจะล็อกช่วงเวลานี้ไม่ให้ผู้อื่นจองซ้ำ
- Rejected (ไม่อนุมัติ) — ผู้ดูแลปฏิเสธคำขอจอง พร้อมบันทึกเหตุผลในฟิลด์ `admin_comment`
- Cancelled (ยกเลิก) — รายการจองถูกยกเลิกโดยผู้จองเอง (ก่อนถึงเวลาเริ่ม) หรือยกเลิกโดยผู้ดูแลระบบ

> **กฎการจองเบื้องต้น 7 ข้อ (SRS ข้อ 13) ที่บังคับใช้ใน `js/domain/rules.js` :**
>
> 1. ผู้ใช้ต้องเลือกห้อง วันที่ เวลาเริ่มต้น และเวลาสิ้นสุดให้ครบถ้วน
> 2. เวลาเริ่มต้นต้องน้อยกว่าเวลาสิ้นสุดเสมอ (`toMin(start_time) < toMin(end_time)`) และห้ามจองย้อนหลัง
> 3. ไม่สามารถจองห้องที่อยู่ในสถานะ `Maintenance` (ปิดปรับปรุง) หรือ `Closed` (ปิดให้บริการ) ได้
> 4. ไม่สามารถจองซ้ำกับรายการที่มีสถานะ `Approved` ในห้องเดียวกัน วันเดียวกัน และช่วงเวลาทับซ้อนกันได้
> 5. คำขอจองใหม่ทุกใบมีสถานะเริ่มต้นเป็น `Pending` เสมอ
> 6. ผู้ใช้สามารถยกเลิกคำขอจองของตนเองได้ เฉพาะเมื่อสถานะเป็น `Pending` หรือ `Approved` และยังไม่ถึงเวลาเริ่มใช้งาน
> 7. Admin สามารถอนุมัติ ปฏิเสธ (ต้องระบุเหตุผล) หรือยกเลิกคำขอจองได้

2.4 เงื่อนไขการตรวจสอบเวลาซ้ำซ้อน (Time Overlap Logic — SRS ข้อ 14)

หัวใจสำคัญที่สุดของระบบจองห้องปฏิบัติการคือการป้องกันการจองเวลาทับซ้อนกันในห้องเดียวกันและวันเดียวกัน โดยระบบจะแปลงเวลาจากสตริง `"HH:MM"` เป็นจำนวนนาทีนับจากเที่ยงคืนด้วยฟังก์ชัน `toMin(t) = h * 60 + m` แล้วตรวจสอบด้วยอสมการคณิตศาสตร์ตามข้อกำหนด SRS ข้อ 14 ดังนี้:

```
new_start_time < existing_end_time   AND   new_end_time > existing_start_time
```

ตารางที่ 2.3 ตัวอย่างการตรวจสอบช่วงเวลาซ้ำซ้อนตาม SRS ข้อ 14 และกรณีขอบเพิ่มเติม

รายการเดิม (Approved)	รายการใหม่ที่ขอจอง	การคำนวณ (นาที)	ผลลัพธ์	คำอธิบาย
`09:00-11:00` (540–660)	`10:00-12:00` (600–720)	`600 < 660 && 720 > 540`	ซ้ำซ้อน (Overlap)	เวลาเริ่มใหม่ (10:00) เข้ามาก่อนเวลาจบเดิม (11:00)
`09:00-11:00` (540–660)	`08:00-09:30` (480–570)	`480 < 660 && 570 > 540`	ซ้ำซ้อน (Overlap)	เวลาจบใหม่ (09:30) ล้ำเข้าไปในเวลาเริ่มเดิม (09:00)
`09:00-11:00` (540–660)	`11:00-12:00` (660–720)	`660 < 660` (False)	ไม่ซ้ำซ้อน	ช่วงเวลาต่อกันพอดี (Back-to-Back) จองต่อได้ทันที

รายการเดิม (Approved)	รายการใหม่ที่ขอจอง	การคำนวณ (นาที)	ผลลัพธ์	คำอธิบาย
`13:00-15:00` (780–900)	`09:00-12:00` (540–720)	`540 < 900 && 720 > 780 (False)`	ไม่ซ้ำซ้อน	อยู่คนละช่วงเวลาอย่างชัดเจน (เช้า vs บ่าย)
`09:00-11:00` (540–660)	`08:30-11:30` (510–690)	`510 < 660 && 690 > 540`	ซ้ำซ้อน (Overlap)	ช่วงเวลาใหม่ครอบคลุมช่วงเวลาเดิมทั้งหมด

2.5 Workflow การทำงานของระบบ (SRS ข้อ 15)

15.1 Workflow การจองห้องของ User

1. เปิดหน้าเว็บ → 2. ดูรายการห้อง → 3. เลือกห้องที่ต้องการ → 4. กรอกฟอร์มจอง →
5. Validate ข้อมูล → 6. ตรวจซ้ำซ้อนกับ Approved → 7. บันทึกเป็น Pending →
8. ติดตามสถานะที่ "การจองของฉัน"

15.2 Workflow การอนุมัติคำขอจองของ Admin

1. Admin Login & เปิดจัดการการจอง → 2. ดูรายการคำขอทั้งหมด →
3. ตรวจสอบรายละเอียด/คำเตือนทับซ้อน → 4. กดอนุมัติ หรือ ปฏิเสธ (ระบุเหตุผล) →
5. อัปเดตสถานะใน IndexedDB → 6. แสดงผลในตารางการจองทันที

15.3 Workflow การดูตารางการจอง (Booking Schedule)

1. เปิดหน้าตารางการจอง (`#/schedule`) → 2. เลือกมุมมอง (รายวัน / รายสัปดาห์) และวันที่ →
3. กรองตามห้องหรือสถานะ → 4. คลิกที่บล็อกเวลาเพื่อดูรายละเอียดใน Modal

CHAPTER 3 · SYSTEM IMPLEMENTATION & CORE CODE EXPLANATION

บทที่ 3 การพัฒนาระบบและอธิบายโค้ดหลัก

3.1 เทคโนโลยีที่ใช้พัฒนา (Technology Stack) และโครงสร้างไฟล์โปรเจกต์

- **Frontend Architecture:** HTML5, CSS3 (Custom Properties + Responsive Grid/Flexbox), Vanilla JavaScript (ES2022 Modules)
- **Data Storage: IndexedDB** (ฐานข้อมูลหลักในเบราว์เซอร์ชื่อ `LabBookingDB`) ร่วมกับ **LocalStorage** สำหรับเก็บ Session และ Profile
- **Data Visualization:** Chart.js 4.4.1 สำหรับแสดงแผนภูมิ Doughnut Chart ในหน้า Admin Dashboard
- **Testing Environment:** Node.js Built-in Assert (`node:assert/strict`) สำหรับรัน Unit Tests แบบอัตโนมัติ

```
c:\mini wed\
├── index.html                # โครงสร้างหลักของหน้าเว็บ (SPA Shell, Topbar, Container, Toast root)
├── package.json              # สคริปต์รันเซิร์ฟเวอร์ (npm start) และรันชุดทดสอบ (npm test)
├── README.md                 # เอกสารแนะนำการติดตั้ง การใช้งาน และอธิบายสถาปัตยกรรม
├── css/
│   └── style.css             # Design System, ตัวแปรสี, ตารางเวลา, Modal และ Responsive Breakpoints
├── js/
│   ├── app.js                # จุดเริ่มต้นแอป, ตั้งค่า Route, เมนูตามบทบาท และ Route Guard (adminOnly)
│   ├── router.js             # Hash Router (#/path/:param?query)
│   ├── db.js                 # IndexedDB Wrapper (Object Stores: rooms, bookings)
│   ├── session.js            # จัดการสถานะ Admin Login และข้อมูลโปรไฟล์ผู้จองปัจจุบัน
│   ├── seed.js               # ข้อมูลตัวอย่างเริ่มต้น (ห้องปฏิบัติการ 5 ห้อง + การจอง 16 รายการ)
│   ├── domain/
│   │   ├── constants.js      # ค่าคงที่สถานะการจอง สถานะห้อง และประเภทผู้จอง
│   │   └── rules.js          # ⭐ กฎธุรกิจหลัก: isOverlap, findConflicts, validateBooking,
canUserCancel
│   ├── services/
│   │   ├── roomService.js    # บริการ CRUD ห้องปฏิบัติการ และตรวจสอบก่อนลบห้อง
│   │   └── bookingService.js # บริการสร้าง อนุมัติ ปฏิเสธ ยกเลิก ค้นหา และคำนวณสถิติการจอง
│   ├── components/
│   │   ├── roomCard.js       # คอมโพเนนต์การ์ดแสดงข้อมูลห้องปฏิบัติการ
│   │   └── bookingDetail.js  # คอมโพเนนต์ Modal แสดงรายละเอียดการจองครบทุกฟิลด์
│   ├── pages/                # โมดูลหน้าจอทั้ง 10 หน้า (home, roomList, roomDetail, bookingForm,
│   │                         # myBookings, schedule, login, adminBookings, adminRooms, dashboard)
│   └── utils/                # ฟังก์ชันช่วยเหลือด้านเวลา (time.js), การแสดงผล (format.js), DOM (dom.js)
└── tests/
    ├── overlap.test.js       # ชุดทดสอบ Unit Test สำหรับ rules.js (29 เทสต์)
    └── service.test.js       # ชุดทดสอบ Integration/Service Test สำหรับ Service Layer (19 เทสต์)
```

3.2 อธิบายโค้ดหลักส่วนที่ 1: กฎธุรกิจและการตรวจสอบเวลาซ้ำซ้อน (`js/domain/rules.js`)

ไฟล์ `js/domain/rules.js` เป็น Pure Functions ที่ไม่พึ่งพา DOM หรือฐานข้อมูล ทำให้สามารถนำไปทดสอบและอธิบายการทำงานได้อย่างชัดเจน:

```
// 1. ฟังก์ชันแปลงเวลา "HH:MM" เป็นจำนวนนาทีนับจาก 00:00 (จาก js/utils/time.js)
export function toMin(t) {
  const [h, m] = String(t).split(':').map(Number);
  return h * 60 + (m || 0); // เช่น "09:30" -> 9*60 + 30 = 570 นาที
}

// 2. ฟังก์ชันตรวจสอบว่าช่วงเวลาใหม่ทับซ้อนกับช่วงเวลาเดิมหรือไม่ (SRS ข้อ 14)
export function isOverlap(newStart, newEnd, existingStart, existingEnd) {
  return toMin(newStart) < toMin(existingEnd) && toMin(newEnd) > toMin(existingStart);
}

// 3. ฟังก์ชันค้นหารายการจองที่ Approved แล้วซึ่งทับซ้อนกับคำขอใหม่ในห้องและวันเดียวกัน
export function findConflicts(bookings, { room_id, booking_date, start_time, end_time, excludeId =
null }) {
  return bookings.filter(
    (b) =>
      b.status === 'Approved' &&
      b.id !== excludeId &&
      b.room_id === room_id &&
      b.booking_date === booking_date &&
      isOverlap(start_time, end_time, b.start_time, b.end_time)
  );
}
```

จุดสำคัญของการออกแบบ: พารามิเตอร์ `excludeId` ถูกออกแบบมาเพื่อรองรับขั้นตอนที่ Admin กดปุ่ม **"อนุมัติ"** รายการที่อยู่ในสถานะ `Pending` โดยระบบจะเรียก `findConflicts` ซ้ำอีกครั้งโดยไม่นับตัวมันเอง เพื่อป้องกันกรณีที่มีคำขอ `Pending` 2 รายการในเวลาเดียวกันแล้ว Admin เผลอกดอนุมัติทั้งคู่

3.3 อธิบายโค้ดหลักส่วนที่ 2: บริการจัดการการจอง (`js/services/bookingService.js`)

`bookingService.js` ทำหน้าที่เชื่อมโยงกฎจาก `rules.js` เข้ากับฐานข้อมูล `IndexedDB` โดยบังคับตรวจ Validation และตรวจเวลาซ้ำซ้อนทั้งตอนสร้างคำขอ (`create`) และตอนอนุมัติ (`approve`):

```
export const bookingService = {
  /** สร้างคำขอจอง: 1) Validate ข้อมูล -> 2) ตรวจเวลาซ้ำซ้อน -> 3) บันทึกสถานะเริ่มต้นเป็น Pending */
  async create(input) {
    const room = await roomService.get(input.room_id);
    const errors = validateBooking(input, room);
    if (Object.keys(errors).length) return { ok: false, errors };

    const conflicts = findConflicts(await this.list(), input);
    if (conflicts.length) {
      return { ok: false, errors: { start_time: conflictMessage(conflicts) }, conflicts };
    }

    const now = new Date().toISOString();
    const booking = {
      id: uid('bk'),
      room_id: input.room_id,
      requester_name: input.requester_name.trim(),
      requester_email: input.requester_email.trim(),
      requester_type: input.requester_type,
      booking_date: input.booking_date,
      start_time: input.start_time,
      end_time: input.end_time,
      purpose: input.purpose.trim(),
      number_of_users: Number(input.number_of_users),
      status: 'Pending',
      admin_comment: '',
      created_at: now,
      updated_at: now,
    };
    await db.put('bookings', booking);
    return { ok: true, booking };
  },

  /** Admin อนุมัติ — ตรวจทับซ้อนซ้ำอีกครั้งก่อนเปลี่ยนสถานะเป็น Approved */
  async approve(id) {
    const b = await this.get(id);
    if (!b) return { ok: false, error: 'ไม่พบรายการจอง' };
    if (b.status !== 'Pending') return { ok: false, error: 'อนุมัติได้เฉพาะรายการที่รออนุมัติ' };
    const conflicts = findConflicts(await this.list(), { ...b, excludeId: b.id });
    if (conflicts.length) return { ok: false, error: conflictMessage(conflicts) };
    const updated = await this._update(id, { status: 'Approved', admin_comment: '' });
    return { ok: true, booking: updated };
  },
};
```

3.4 ข้อมูลตัวอย่างห้องปฏิบัติการและการจองในระบบ (SRS ข้อ 11 และ ข้อ 23.6–23.7)

ตามข้อกำหนดการส่งงาน (SRS ข้อ 23) ระบบต้องมีข้อมูลห้องปฏิบัติการอย่างน้อย 3 ห้อง และข้อมูลการจองตัวอย่างอย่างน้อย 10 รายการ โดยในไฟล์ `js/seed.js` ได้จัดเตรียม **ห้องปฏิบัติการครบทั้ง 5 ห้อง** และ **รายการจองตัวอย่าง 16 รายการ** ดังนี้:

ตารางที่ 3.1 ข้อมูลห้องปฏิบัติการตัวอย่างทั้ง 5 ห้องในระบบ (`buildSeedRooms`)

รหัสห้อง	ชื่อห้องปฏิบัติการ	ความจุ	อุปกรณ์หลัก	สถานที่ตั้ง	สถานะ
CE-LAB-01	Computer Engineering Lab 1	30 คน	Computer, Projector, Internet	อาคารวิศวกรรมศาสตร์ ชั้น 2 ห้อง 201	Available
CE-LAB-02	Computer Engineering Lab 2	25 คน	Computer, Smart TV, Internet	อาคารวิศวกรรมศาสตร์ ชั้น 2 ห้อง 202	Available
NET-LAB	Network Laboratory	20 คน	Router, Switch, Network Cable, Rack	อาคารวิศวกรรมศาสตร์ ชั้น 3 ห้อง 305	Available
IOT-LAB	IoT and Embedded Systems Lab	20 คน	Arduino, ESP32, Sensor Kit, Oscilloscope	อาคารวิศวกรรมศาสตร์ ชั้น 3 ห้อง 306	Available
PROJECT-ROOM	Student Project Room	10 คน	Whiteboard, Meeting Table, Power Outlet	อาคารวิศวกรรมศาสตร์ ชั้น 1 ห้อง 105	Available

ตารางที่ 3.2 ข้อมูลการจองตัวอย่าง 16 รายการในระบบ (`buildSeedBookings` — อิงวันที่สัมพัทธ์จากวันที่เปิดใช้งาน)

รหัส	ห้อง	วันที่	ช่วงเวลา	ผู้จอง (บทบาท)	วัตถุประสงค์การใช้งาน	สถานะ
`bk-01`	CE-LAB-01	วันนี้	09:00–11:00	อ.สุดา (Teacher)	สอนปฏิบัติการวิชาโครงสร้างข้อมูล (28 คน)	Approved
`bk-02`	CE-LAB-01	วันนี้	13:00–15:00	สมชาย (Student)	ติวสอบกลางภาควิชา Programming (20 คน)	Pending
`bk-03`	CE-LAB-01	วันนี้	13:30–14:30	อนันต์ (Student)	ซ้อมนำเสนอโปรเจกต์ (6 คน — ทดสอบ Pending ซ้อนกัน)	Pending
`bk-04`	CE-LAB-01	+1 วัน	10:00–12:00	อ.วิชัย (Teacher)	สอบปฏิบัติวิชาฐานข้อมูล (30 คน)	Approved

รหัส	ห้อง	วันที่	ช่วงเวลา	ผู้จอง (บทบาท)	วัตถุประสงค์การใช้งาน	สถานะ
`bk-05`	CE-LAB-01	+3 วัน	08:30–10:30	มาลี (Student)	อบรมการใช้ Git และ GitHub (15 คน)	Pending
`bk-06`	CE-LAB-01	+4 วัน	13:00–16:00	อ.สุดา (Teacher)	สอนชดเชยวิชาอัลกอริทึม (25 คน)	Approved
`bk-07`	CE-LAB-02	วันนี้	10:00–12:00	พิมพ์ใจ (Staff)	อบรมระบบสารสนเทศสำหรับเจ้าหน้าที่ (20 คน)	Approved
`bk-08`	CE-LAB-02	+1 วัน	13:00–16:00	สมชาย (Student)	เวิร์กช็อป Web Development (18 คน)	Pending
`bk-09`	CE-LAB-02	-1 วัน	13:00–15:00	อนันต์ (Student)	ทำงานกลุ่มวิชา Software Engineering (12 คน)	Rejected
`bk-10`	NET-LAB	+1 วัน	09:00–12:00	อ.วิชัย (Teacher)	Workshop Cisco Packet Tracer (20 คน)	Approved
`bk-11`	NET-LAB	+2 วัน	13:00–15:30	มาลี (Student)	ฝึกตั้งค่า VLAN สำหรับสอบ CCNA (10 คน)	Rejected
`bk-12`	NET-LAB	-2 วัน	09:00–11:00	อ.สุดา (Teacher)	สอนปฏิบัติวิชาเครือข่ายคอมพิวเตอร์ (18 คน)	Approved
`bk-13`	IOT-LAB	+2 วัน	09:00–12:00	อ.วิชัย (Teacher)	ปฏิบัติการ ESP32 กับ MQTT (20 คน)	Approved
`bk-14`	IOT-LAB	+3 วัน	14:00–16:00	สมชาย (Student)	ทดสอบเซนเซอร์โปรเจกต์ Smart Farm (5 คน)	Pending
`bk-15`	PROJECT-ROOM	วันนี้	14:00–16:00	อนันต์ (Student)	ประชุมทีมโปรเจกต์จบ (8 คน)	Approved
`bk-16`	PROJECT-ROOM	+1 วัน	09:00–10:30	มาลี (Student)	ติวหนังสือกลุ่ม (6 คน — ผู้จองยกเลิกเอง)	Cancelled

CHAPTER 4 · SYSTEM TESTING & USER MANUAL

บทที่ 4 การทดสอบระบบและคู่มือการใช้งาน

4.1 การทดสอบระดับหน่วยและระดับบริการ (Automated Unit & Service Tests)

ระบบมีชุดทดสอบอัตโนมัติในโฟลเดอร์ `tests/` จำนวนรวม **48 ชุดทดสอบ** แบ่งออกเป็น `tests/overlap.test.js` (29 เทสต์) และ `tests/service.test.js` (19 เทสต์) ซึ่งผลการรันผ่านครบทุกข้อดังนี้:

```
$ node tests/overlap.test.js && node tests/service.test.js
isOverlap — ตัวอย่างตาม SRS ข้อ 14
  ✓ 09:00-11:00 vs 10:00-12:00 => ซ้ำซ้อน
  ✓ 09:00-11:00 vs 08:00-09:30 => ซ้ำซ้อน
  ✓ 09:00-11:00 vs 11:00-12:00 => ไม่ซ้ำซ้อน (ต่อกันพอดี)
  ✓ 13:00-15:00 vs 09:00-12:00 => ไม่ซ้ำซ้อน
isOverlap — กรณีขอบ (เวลาเหมือนกัน, ช่วงใหม่ครอบช่วงเดิม, ช่วงใหม่อยู่ในช่วงเดิม, ต่อกันด้านหน้า, เทียบตัวเลข)
  ✓ ผ่านครบ 5 กรณีขอบ
findConflicts & validateBooking & canUserCancel & countByStatus
  ✓ ผ่านครบ 20 กรณีทดสอบ (รวมผ่านทั้งหมด 29 เทสต์)

seed / create / approve / reject / cancel / search / rooms (service.test.js)
  ✓ มีห้อง >= 3 และการจอง >= 10, ข้อมูลตัวอย่างไม่มี Approved ทับซ้อนกันเอง
  ✓ จองสำเร็จ => สถานะ Pending, จองชนกับ Approved => ถูกปฏิเสธ, จองต่อกันพอดี => สำเร็จ
  ✓ อนุมัติรายการที่ชนกับ Approved ไม่ได้, ปฏิเสธต้องมีเหตุผล, User/Admin ยกเลิกได้ตามสิทธิ์
  ✓ ค้นหา/กรองตามห้อง วันที่ สถานะ และ CRUD ห้องปฏิบัติการ (รวมผ่านทั้งหมด 19 เทสต์)
```

4.2 การตรวจสอบตามเกณฑ์ตรวจรับระบบ (Acceptance Criteria — SRS ข้อ 20)

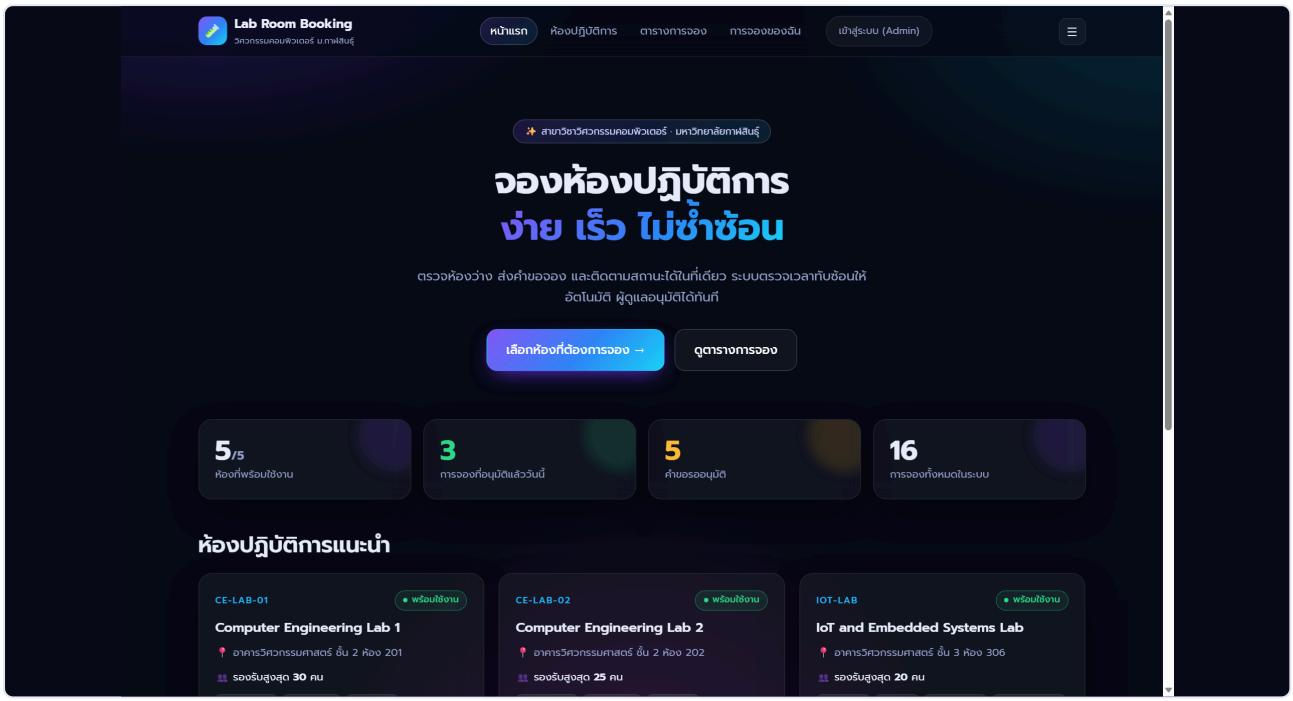
ตารางที่ 4.1 ผลการตรวจรับระบบเทียบกับเกณฑ์ขั้นต่ำ 13 ข้อ ตาม SRS ข้อ 20

ข้อ	เกณฑ์การตรวจรับ (Acceptance Criteria)	สถานะ	หลักฐาน / การทำงานในระบบ
1	ผู้ใช้สามารถดูรายการห้องปฏิบัติการได้	✓ ผ่าน	แสดงการ์ดห้องพร้อมตัวกรองและช่องค้นหาในหน้า `#/rooms`
2	ผู้ใช้สามารถส่งคำขอจองห้องได้	✓ ผ่าน	แบบฟอร์มจองห้องที่หน้า `#/book` พร้อม Validation ครบทุกฟิลด์
3	ระบบสามารถบันทึกข้อมูลการจองได้	✓ ผ่าน	บันทึกลงฐานข้อมูล IndexedDB (`LabBookingDB.bookings`)
4	ระบบตรวจสอบการจองซ้ำซ้อนในห้องและเวลาทับซ้อนได้	✓ ผ่าน	ตรวจสอบด้วย `isOverlap` และแจ้งเตือนแบบ Real-time บนฟอร์ม
5	ไม่อนุญาตให้จองห้องที่เวลาทับซ้อนกับรายการที่ Approved แล้ว	✓ ผ่าน	บล็อกทั้งตอนส่งคำขอจอง และบล็อกปุ่มอนุมัติในหน้า Admin
6	Admin สามารถอนุมัติและปฏิเสธคำขอจองได้	✓ ผ่าน	จัดการได้ที่หน้า `#/admin/bookings` พร้อมบังคับใส่เหตุผลเมื่อปฏิเสธ

ข้อ	เกณฑ์การตรวจรับ (Acceptance Criteria)	สถานะ	หลักฐาน / การทำงานในระบบ
7	ระบบสามารถแสดงรายการจองตามสถานะได้	✓ ผ่าน	มีแท็บกรองตามสถานะทั้งในหน้า `#/my` และ `#/admin/bookings`
8	ระบบสามารถค้นหาและกรองรายการจองได้	✓ ผ่าน	ค้นหาตามชื่อ/อีเมล/วัตถุประสงค์ และกรองตามห้อง วันที่ สถานะ
9	ระบบมีตารางการจองรายวันหรือรายสัปดาห์	✓ ผ่าน	หน้า `#/schedule` สลับมุมมองรายวัน (แยกตามห้อง) และรายสัปดาห์ได้
10	ระบบมี Dashboard สรุปจำนวนการจองตามสถานะ	✓ ผ่าน	หน้า `#/admin/dashboard` แสดงตัวเลขสรุป กราฟวงกลม และกราฟแท่ง
11	ระบบมี UI ที่ใช้งานง่ายและรองรับมือถือ	✓ ผ่าน	ออกแบบ Responsive CSS รองรับทั้ง Desktop, Tablet และ Mobile
12	ระบบถูก Deploy และเข้าถึงผ่าน URL ได้จริง	✓ ผ่าน	เชื่อมต่อ GitHub Repository `Newphakhin/esfsdfsdf` พร้อม Deploy
13	มี README อธิบายวิธีติดตั้ง วิธีใช้งาน และเครื่องมือ AI ที่ใช้	✓ ผ่าน	จัดทำ `README.md` และเล่มรายงานฉบับสมบูรณ์เรียบร้อย

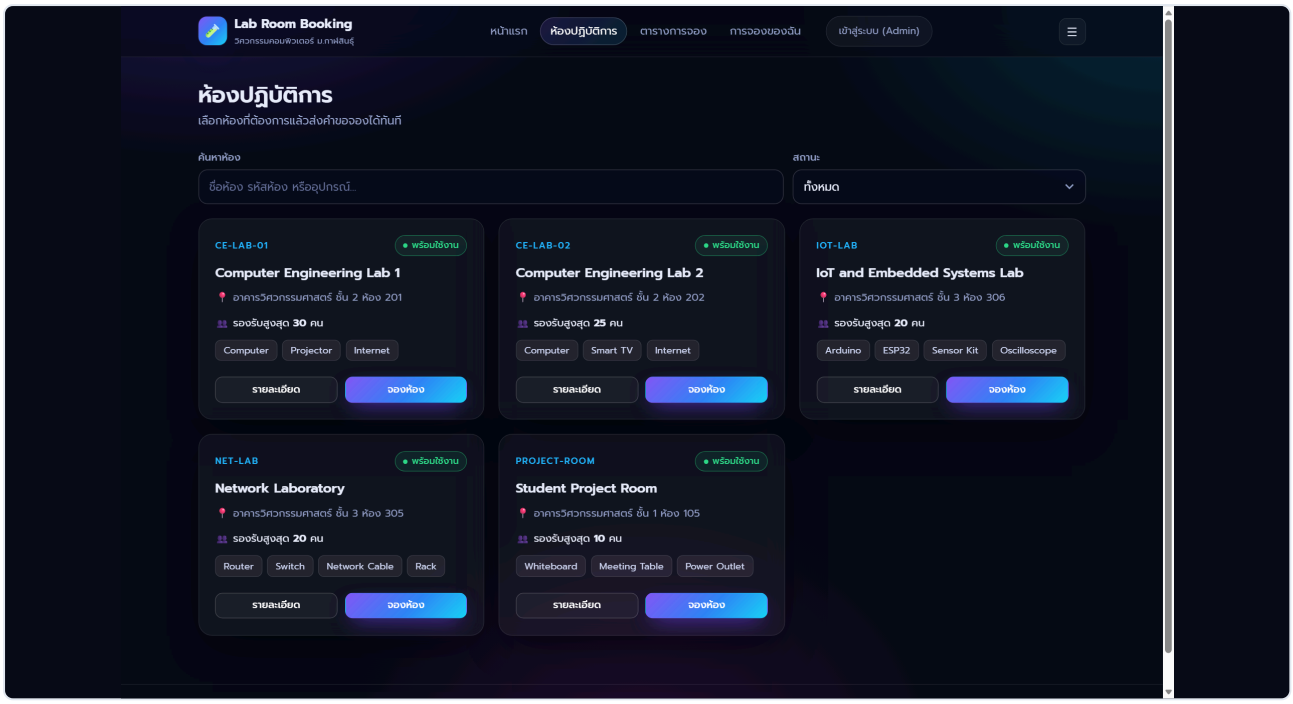
4.3 คู่มือการใช้งานและภาพหน้าจอหลักของระบบ (SRS ข้อ 16 และ ข้อ 23.4)

ระบบประกอบด้วยหน้าจอหลักครบทั้ง 10 หน้าจอ ตามข้อกำหนด SRS ข้อ 16 โดยแสดงภาพหน้าจอจากการทำงานจริงของระบบดังนี้:



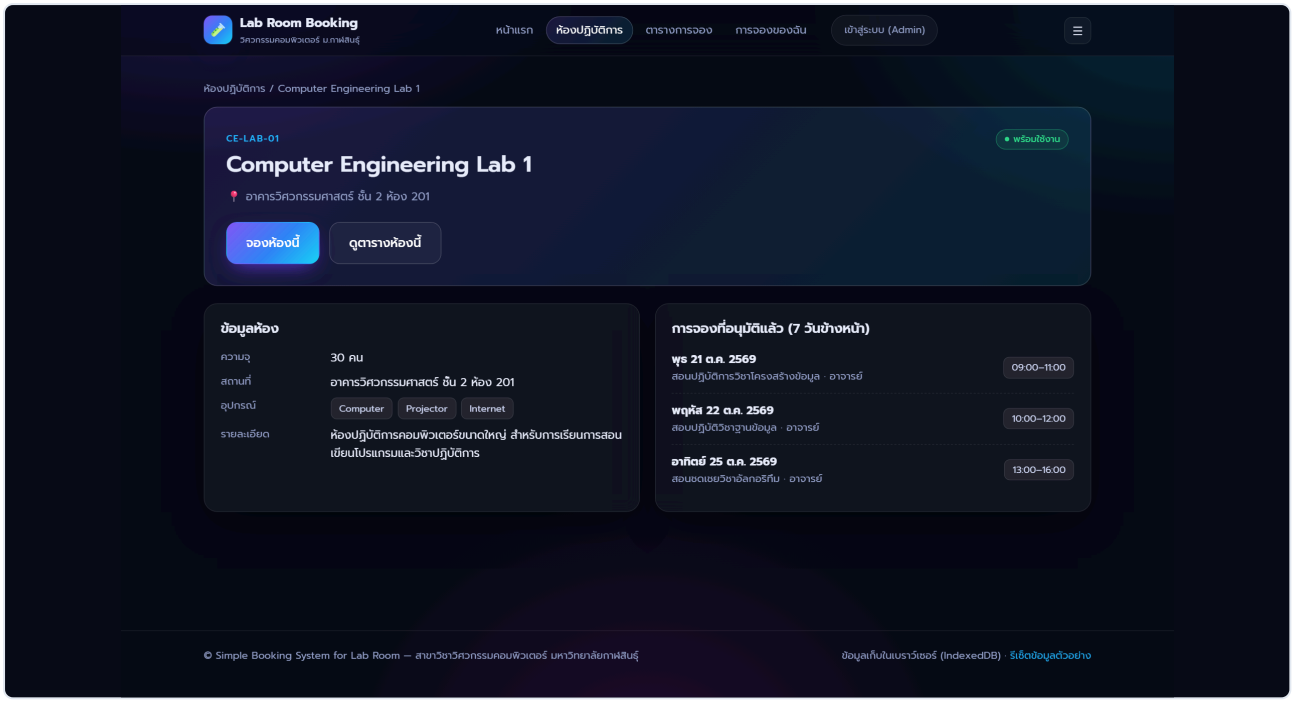
ภาพที่ 4.1 หน้าแรกของระบบ (Home Page — `#/`)

แสดงแบนเนอร์แนะนำระบบ ปุ่มลัดเข้าสู่หน้าจองห้องและตารางการจอง การ์ดสรุปภาพรวม (ห้องที่พร้อมใช้งาน การจองวันนี้ คำขอรออนุมัติ) และรายการห้องปฏิบัติการแนะนำ



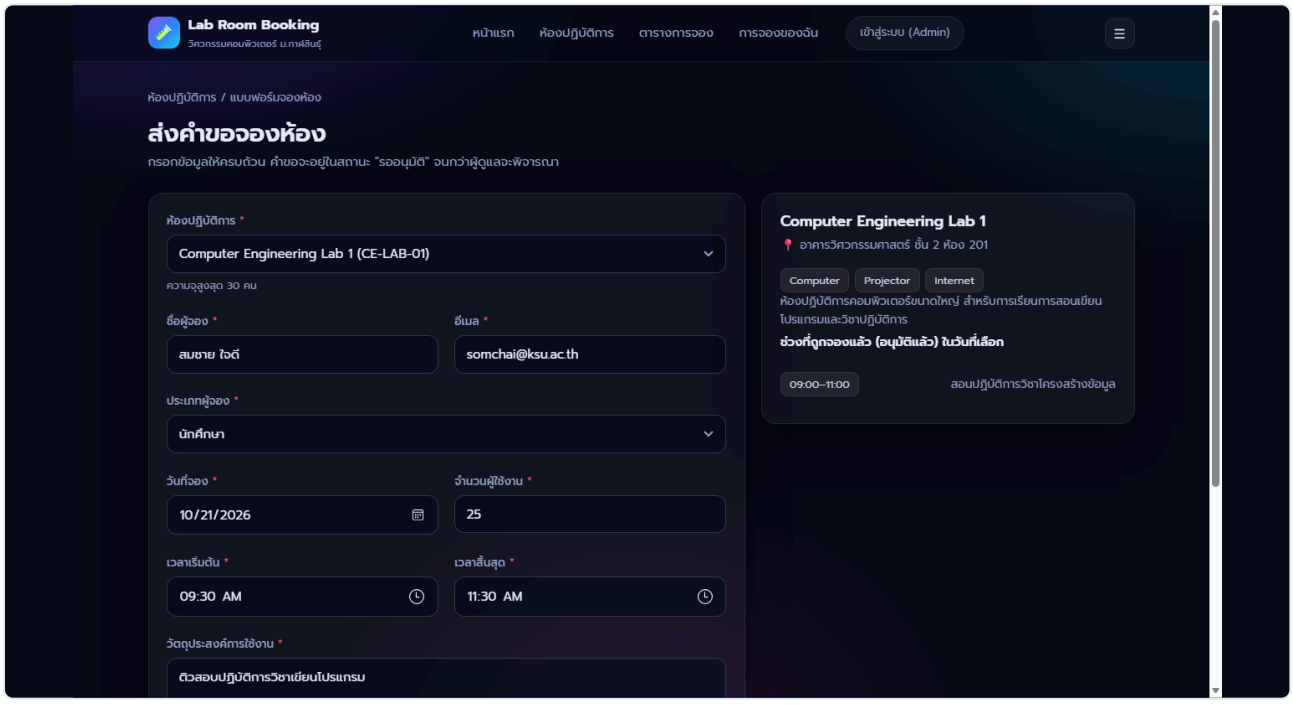
ภาพที่ 4.2 หน้ารายการห้องปฏิบัติการทั้งหมด (Lab Room List Page — `#/rooms`)

แสดงการ์ดห้องปฏิบัติการทั้ง 5 ห้อง พร้อมรหัสห้อง ชื่อห้อง สถานที่ตั้ง ความจุ รายการอุปกรณ์ สถานะห้อง และช่องค้นหา/กรองตามสถานะแบบทันที



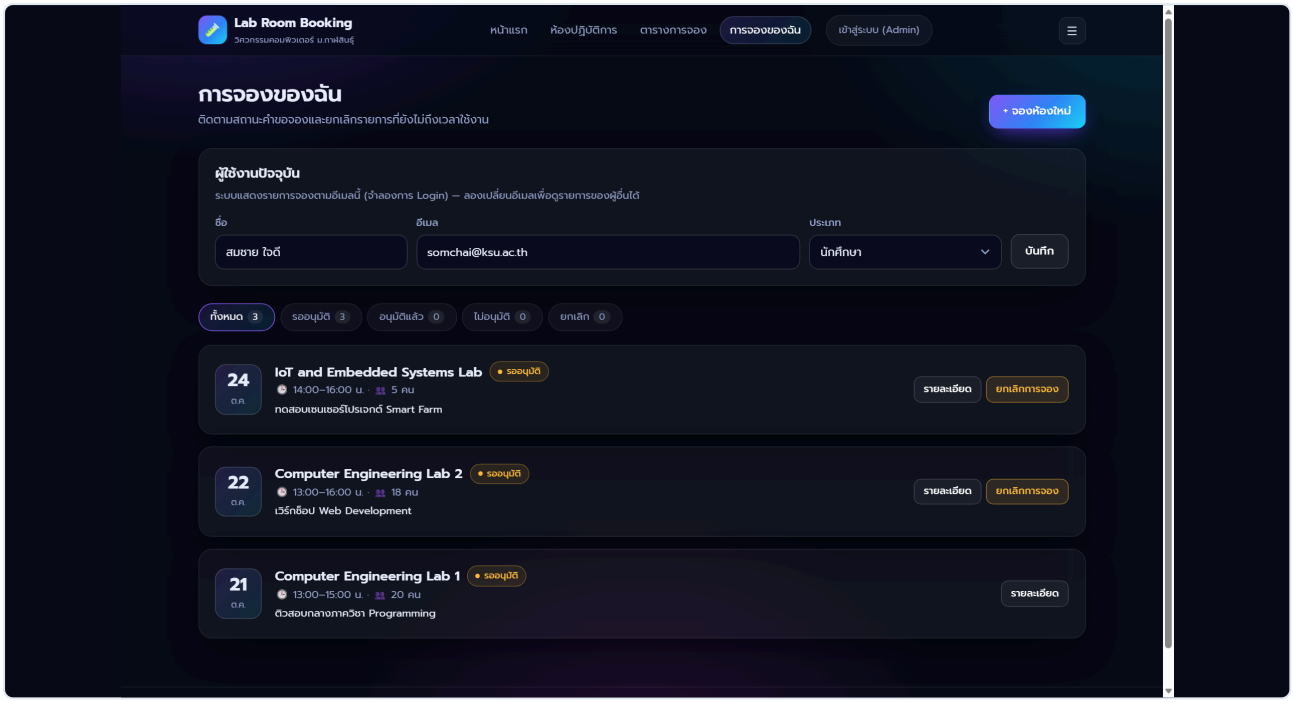
ภาพที่ 4.3 หน้ารายละเอียดห้องปฏิบัติการ (Lab Room Detail Page — `#/rooms/room-ce1`)

แสดงข้อมูลเชิงลึกของห้องปฏิบัติการ อุปกรณ์ทั้งหมด คำอธิบายห้อง และรายการจองที่ได้รับการอนุมัติแล้วในช่วง 7 วันข้างหน้า พร้อมปุ่มกดจองห้อง



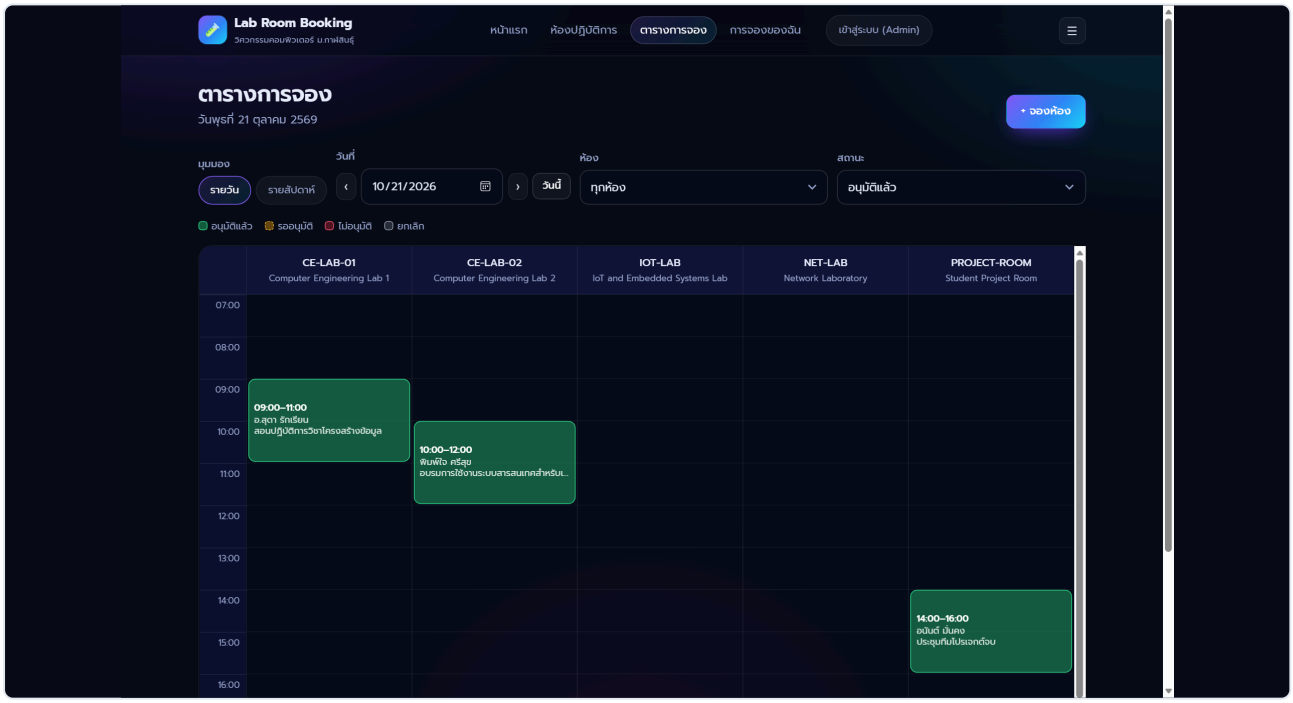
ภาพที่ 4.4 หน้าแบบฟอร์มส่งคำขอจองห้องและระบบแจ้งเตือนเวลาทับซ้อนแบบ Real-time (`#/book`)

เมื่อผู้ใช้เลือกห้อง วันที่ และช่วงเวลา (เช่น 09:30–11:30 น. ซึ่งชนกับรายการที่อนุมัติแล้วช่วง 09:00–11:00 น.) ระบบจะแสดงกล่องแจ้งเตือนสีเหลืองทันทีพร้อมแสดงช่วงเวลาที่ถูกจองแล้วทางด้านขวา



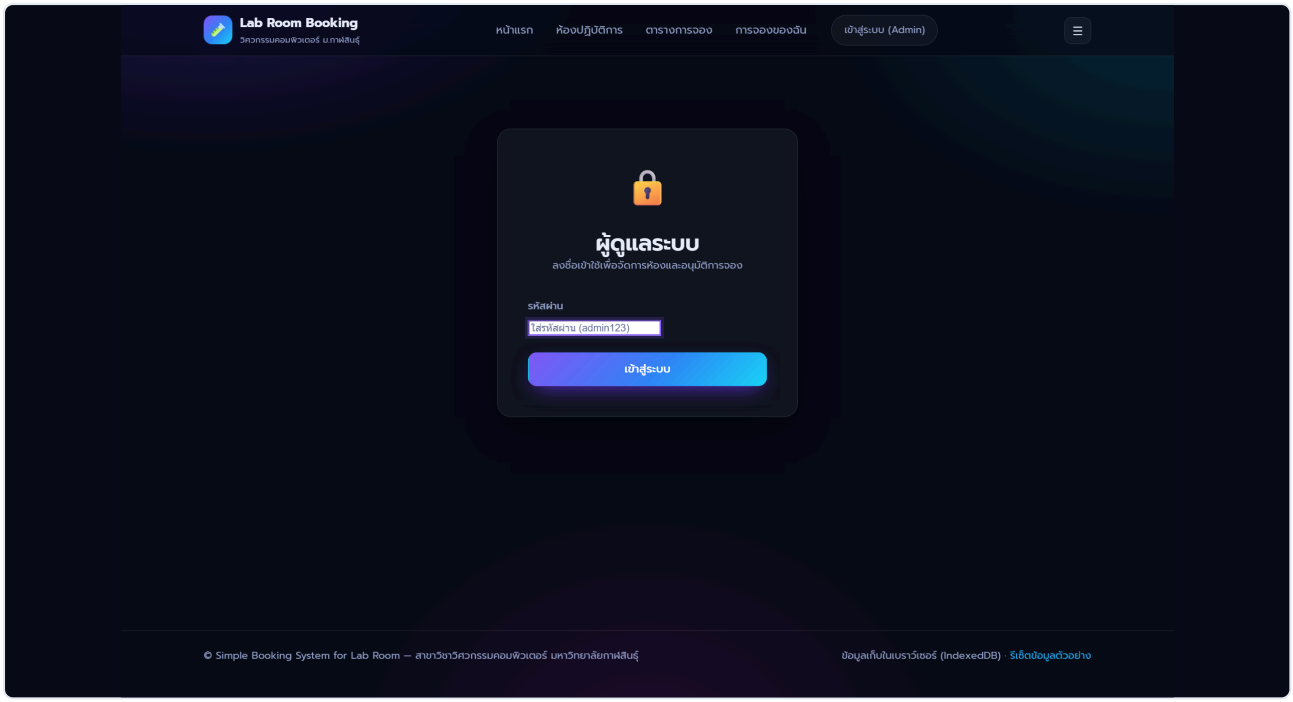
ภาพที่ 4.5 หน้าการจองของฉัน (My Booking Page — `#/my`)

แสดงรายการคำขอจองของผู้ใช้ตามอีเมลที่ระบุ มีแท็บกรองตามสถานะ แสดงเหตุผลจากผู้ดูแลระบบ และมีปุ่ม "ยกเลิกการจอง" สำหรับรายการที่ยังไม่ถึงเวลาเริ่มใช้งาน



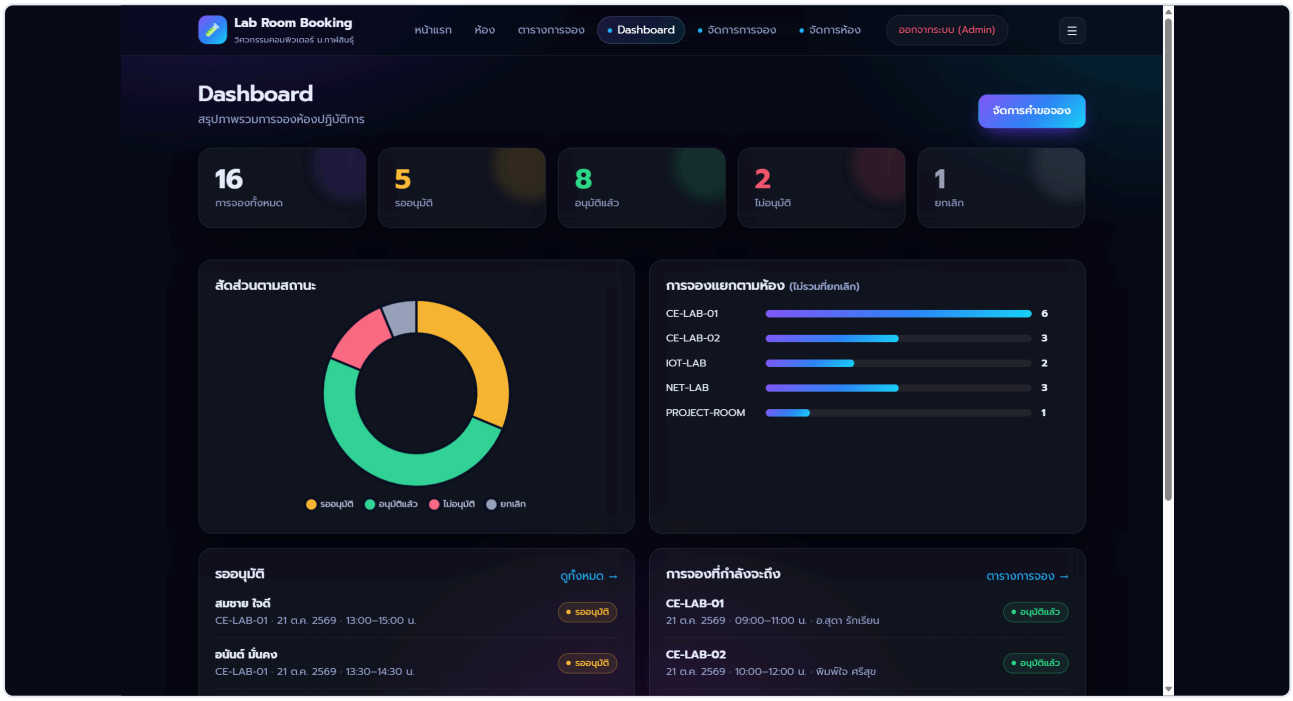
ภาพที่ 4.6 หน้าตารางการจองรายวันและรายสัปดาห์ (Booking Schedule Page — `#/schedule`)

แสดงตารางเวลาแบบ Visual Timeline แยกตามห้องปฏิบัติการ (มุมมองรายวัน) หรือแยกตามวันจันทร์–อาทิตย์ (มุมมองรายสัปดาห์) สามารถกรองตามห้องและสถานะได้



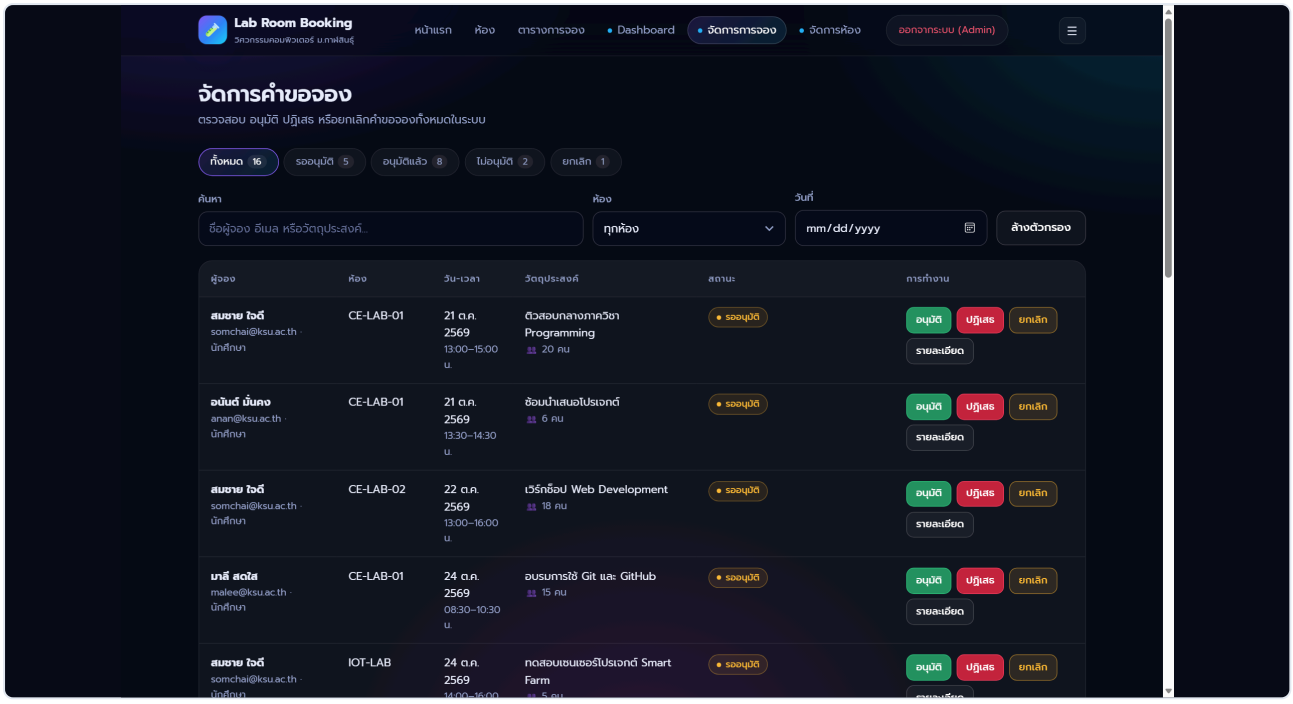
ภาพที่ 4.7 หน้าเข้าสู่ระบบสำหรับผู้ดูแลระบบ (Admin Login Page — `#/login`)

ป้องกันการเข้าถึงหน้าจัดการของ Admin ตามข้อกำหนด NFR-05 โดยผู้ดูแลระบบใช้รหัสผ่าน `admin123` เพื่อเข้าสู่โหมดผู้ดูแลระบบ



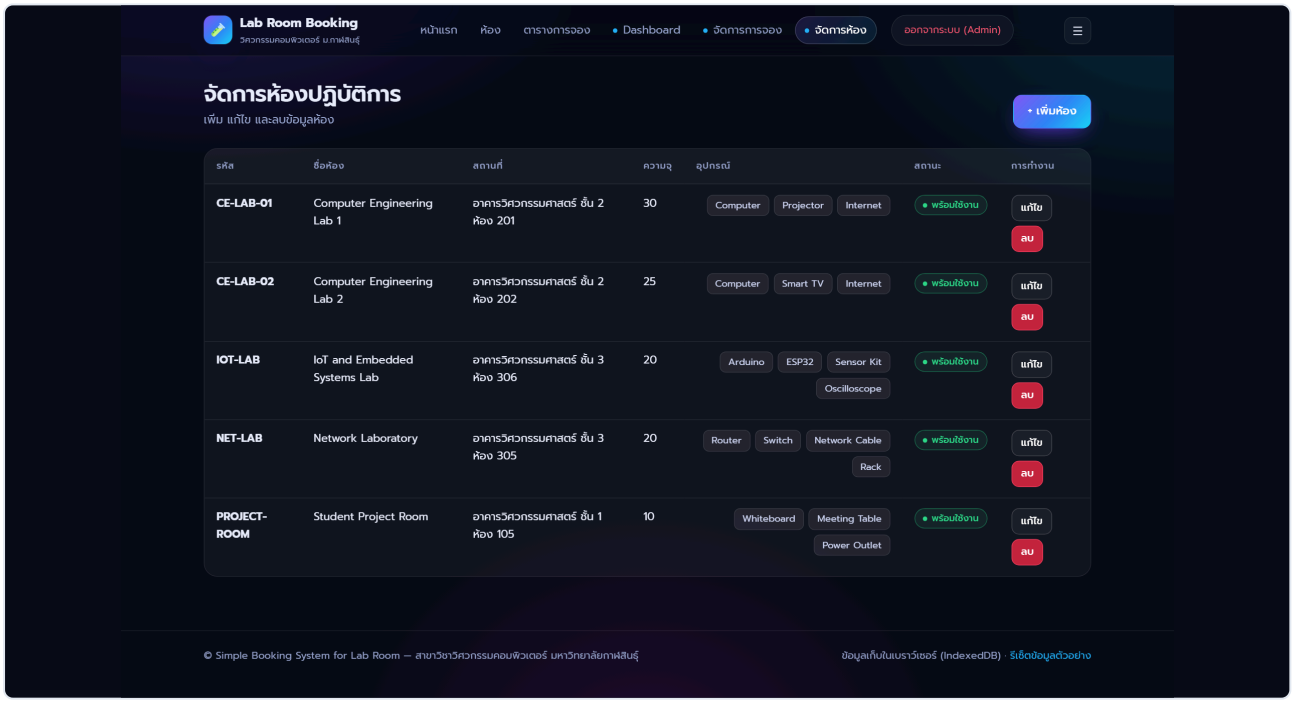
ภาพที่ 4.8 หน้าสรุปข้อมูลผู้ดูแลระบบ (Dashboard Page — `#/admin/dashboard`)

แสดงจำนวนการจองทั้งหมดแยกตามสถานะ แผนภูมิวงกลม (Chart.js Doughnut Chart) แผนภูมิแท่งแสดงจำนวนการจองแยกตามห้อง รายการรออนุมัติล่าสุด และการจองที่กำลังจะถึง



ภาพที่ 4.9 หน้าจัดการคำขอจองของผู้ดูแลระบบ (Admin Booking Management Page — `#/admin/bookings`)

แสดงรายการคำขอจองทั้งหมดพร้อมช่องค้นหา ตัวกรองห้อง/วันที่/สถานะ และปุ่ม "อนุมัติ", "ปฏิเสธ" (พร้อมกล่องกรอกเหตุผล) และ "ยกเลิก"



ภาพที่ 4.10 หน้าจัดการข้อมูลห้องปฏิบัติการ (Admin Room Management Page — `#/admin/rooms`)

รองรับการเพิ่มห้องใหม่ แก้ไขรายละเอียด/ความจุ/อุปกรณ์/สถานะห้อง และลบห้องปฏิบัติการ (มีระบบป้องกันการลบห้องที่ยังมีการจองค้างอยู่)

CHAPTER 5 · CONCLUSION, AI TOOLS USAGE & DEPLOYMENT

บทที่ 5 สรุปผลการดำเนินงานและการใช้ AI Tools ในการพัฒนา

5.1 สรุปผลการพัฒนาโปรเจกต์และผลลัพธ์การเรียนรู้ (SRS ข้อ 18 และ ข้อ 24)

จากการพัฒนาโปรเจกต์ **Simple Booking System for Lab Room** ระบบสามารถทำงานได้ครบถ้วนตามเกณฑ์ **Minimum Viable Product (MVP)** ทั้ง 11 ข้อ และบรรลุผลลัพธ์การเรียนรู้ (Learning Outcomes) ทั้ง 10 ข้อ ตามเอกสาร SRS ได้แก่:

- พัฒนาเว็บแอปพลิเคชันที่มีฟังก์ชัน CRUD ทั้งข้อมูลห้องปฏิบัติการและข้อมูลการจองได้อย่างสมบูรณ์
- ออกแบบตรรกะการตรวจสอบเวลาจองซ้ำซ้อน (Overlap Detection) และจัดการฟอร์มพร้อม Validation ได้อย่างรัดกุม
- พัฒนา Role-based View แยกมุมมองผู้ใช้งานทั่วไป (User) และผู้ดูแลระบบ (Admin) พร้อมระบบป้องกันสิทธิ์เข้าถึง
- สร้าง Dashboard สรุปข้อมูลสถิติและตารางแสดงผลเวลาการจองรายวัน/รายสัปดาห์ที่รองรับหน้าจอมือถือ (Responsive UI)

5.2 สรุปการใช้ AI Tools ในการพัฒนา (ตามข้อกำหนด SRS ข้อ 22 และ ข้อ 23.8)

ในการพัฒนาโปรเจกต์นี้ ผู้จัดทำได้ประยุกต์ใช้เครื่องมือ AI Coding Assistant (**Antigravity**) เพื่อเพิ่มประสิทธิภาพในกระบวนการวิศวกรรมซอฟต์แวร์ โดยผู้จัดทำได้ตรวจสอบ ทดสอบ และทำความเข้าใจโค้ดหลักทั้งหมดด้วยตนเอง ดังสรุปในตารางที่ 5.1:

ตารางที่ 5.1 สรุปบทบาทการใช้ AI Tools ในแต่ละขั้นตอนการพัฒนาโปรเจกต์

ลำดับ	ขั้นตอนตามแนวทาง SRS ข้อ 22	รายละเอียดการใช้ AI Tools ช่วยพัฒนา	การตรวจสอบโดยผู้พัฒนา
1	ออกแบบ Project Structure	ช่วยวางโครงสร้างแบบแยกชั้น (`pages/` , `services/` , `domain/` , `components/` , `utils/`)	ตรวจสอบการแยก Module และการ Import
2	ออกแบบ Database Schema	ช่วยออกแบบโครงสร้าง Object Stores (`rooms` , `bookings`) และ Indexes ใน `js/db.js`	ตรวจสอบฟิลด์ให้ตรงกับ SRS ข้อ 10
3	สร้าง Component & Function	ช่วยเขียน ES Modules สำหรับหน้าเว็บทั้ง 10 หน้า และ Reusable Modal/Toast	ทดสอบการแสดงผลและการเปลี่ยนหน้า
4	เขียน Booking Form Validation	ช่วยเขียนฟังก์ชัน `validateBooking` และ `validateRoom` ตรวจข้อมูลครบทุกเงื่อนไข	ทดสอบกรอกข้อมูลผิดรูปแบบและข้อมูลว่าง

ลำดับ	ขั้นตอนตามแนวทาง SRS ข้อ 22	รายละเอียดการใช้ AI Tools ช่วยพัฒนา	การตรวจสอบโดยผู้พัฒนา
5	เขียน Logic ตรวจสอบเวลาทับซ้อน	ช่วยเขียนฟังก์ชัน `isOverlap` และ `findConflicts` ใน `js/domain/rules.js`	คำนวณมือเปรียบเทียบและรัน Unit Test 29 ข้อ
6	สร้าง Dashboard Layout	ช่วยเชื่อมต่อ Chart.js แสดงกราฟ Doughnut และกราฟแท่งเปรียบเทียบรายห้อง	ตรวจสอบยอดรวมตามสถานะให้ตรงกับข้อมูลจริง
7	Debug Error & Async Flow	ช่วยปรับปรุงชุดทดสอบ `tests/service.test.js` ให้รองรับการทำงานแบบ Async กับ IndexedDB	รันชุดทดสอบผ่านครบ 48 เทสต์
8	สร้าง Sample Data	ช่วยสร้างข้อมูลห้องปฏิบัติการ 5 ห้อง และรายการจองตัวอย่าง 16 รายการใน `js/seed.js`	ตรวจสอบความสมจริงและสถานะการจอง
9	ปรับ UI ให้เป็น Responsive	ช่วยจัดทำ Media Queries ใน `css/style.css` สำหรับแสดงตารางเป็นการ์ดบนมือถือ	ทดสอบย่อ-ขยายหน้าจอบนเบราว์เซอร์
10	จัดทำ README และเล่มรายงาน	ช่วยเรียบเรียงเอกสารคู่มือ `README.md` และจัดทำเล่มรายงานฉบับสมบูรณ์	ตรวจทานเนื้อหาและความถูกต้องทั้งหมด

5.3 การติดตั้ง การใช้งาน และการนำขึ้นระบบจริง (Deployment & Deliverables — SRS ข้อ 23)

ข้อมูลการส่งงาน (Deliverables Summary):

- **ผู้จัดทำ:** นาย ภคิน ภารสมบูรณ์ (สาขาวิชาวิศวกรรมคอมพิวเตอร์ มหาวิทยาลัยกาฬสินธุ์)
- **อาจารย์ประจำวิชา:** อาจารย์ วรพจน์ สมมูล
- **GitHub Repository:** https://github.com/Newphakhin/esfsdfsdf
- **คำสั่งรันระบบในเครื่อง (Local Server):** `npx serve .` หรือ `python -m http.server 5173`
- **คำสั่งรันชุดทดสอบอัตโนมัติ:** `node tests/overlap.test.js && node tests/service.test.js`
- **การ Deploy ขึ้น URL สาธารณะ:** เชื่อมต่อ Repository `Newphakhin/esfsdfsdf` เข้ากับ **GitHub Pages, Vercel หรือ Netlify** โดยตั้งค่า Publish Directory เป็น `.` (Root) ได้ทันทีโดยไม่ต้องใช้คำสั่ง Build